# Batch mode: define a dataset, process it, read it back

Developer quick-sheet · goldsluice and goldpan share one dataset file · version 1, 27 October 2026

A **dataset definition** is one small text file that says *which raw files*, *how to read them* and *which processing to run*. `goldsluice` runs it over every raw file in one command and writes the results next to it; `goldpan` opens the same file and gives you the pulses together with the processed quantities. Nothing is assumed: the readout, the channels and every parameter are written in the file. This sheet is the whole loop; the file format is specified in the docstring of `python/goldpan/dataset.py`, and a new algorithm is a different sheet (*Running goldsluice and developing an algorithm*).

You need the setup of that sheet (`maturin develop --release` in `goldsluice/`) and `goldpan` installed with Parquet support (`pip install -e "./python[parquet]"` from the repository).

## 1. The definition file

A TOML file, by convention `<name>.dataset.toml`. Annotated; the whole file is `examples/r76.dataset.toml`:

```toml
[dataset]
name = "r76-07221203"
output = "~/goldsluice-output/r76" # the processed quantities: outside the repository

[[sources]]                        # where the pulses come from; several allowed
name = "side2"
format = "idx"                     # any goldpan adapter: idx, midas, parquet, ...
glob = "~/idx/07221203_2025_F*"    # raw files or folders: paths = [..] and/or glob
detector = "NAI-S2"
readout = "DCRC_RUN76"             # a goldpan.config preset, or three numbers
[sources.read]                     # the adapter's other read() arguments
channels = { Side2pC = [0, 0], Side2pF = [0, 1] }
roles = { Side2pC = "phonon", Side2pF = "phonon" }

[[processing]]                     # the steps, in order
algorithm = "peak"
channels = "role:phonon"           # "all", "role:<role>", or a list of names
[processing.params]
glitch_samples = "@readout.glitch_samples"       # from the readout above
pretrigger_samples = "@readout.pretrigger_samples"
amps_per_adc = 1.5e-9              # you must supply it: there is no default

[[processing]]                     # the next step, run in order
algorithm = "max_time"             # time of the largest sample, in s
channels = "role:phonon"
[processing.params]
sample_period_s = "@readout.sample_period_s"   # from the readout
```

| Key | Rule |
|---|---|
| `readout` | A preset name or `{ pretrigger_samples = ..., glitch_samples = ..., sample_period_s = ... }`, all three. Only needed by adapters that ask for it (`idx`, `midas`); a `parquet` source has its readout inside the file. |
| `[sources.read]` | Exactly the keyword arguments of that adapter's `read` (`help(goldpan.adapters.idx.read)`); an argument it does not have is an error. |
| `"@readout.<field>"` | In a parameter: the value of `pretrigger_samples`, `glitch_samples` or `sample_period_s` of the data's readout, so the file cannot disagree with itself. |

| | |
|---|---|
| Unknown keys | **Errors**, not ignored: a typo (`readuot`) fails at once instead of silently doing nothing. |

## 2. Check, then run

```
python -m goldsluice algorithms
python -m goldsluice check r76.dataset.toml
python -m goldsluice run r76.dataset.toml
```

`algorithms` lists what can be named in `algorithm =` and the outputs of each. `check` reads the definition and lists the raw files it finds; it fails for an unknown algorithm, a pattern that matches nothing or a bad readout, before anything is processed. `run` processes the files and ends with a report that times each phase:

```
processed  .../raw/TOY/S1_F0001.parquet: 3000 events in 0.75 s
processed  .../raw/TOY/S1_F0002.parquet: 3000 events in 0.65 s
dataset big: 2 file(s) processed, 0 unchanged, 6000 events in 1.49 s
  read        1.244 s   (the adapters)
  compute     0.052 s   (peak)
  write       0.101 s
       4021 events/s overall
```

That is 6000 events of 3 channels and 4096 samples from a Parquet source. **Reading is usually most of the time**; the Rust kernels the least. Options: `--chunk N` (events read at once, default 1000: it bounds the memory used by the pulses), `--force` (redo files that are unchanged) and `--backend python` (the Python mirrors: slow, to compare numbers).

**Results do not belong in the repository.** `check` and `run` refuse an `output` folder that lies inside a Git working tree whose `.gitignore` does not cover the data files written there, and say so before anything is written. The repository ignores `processed/` and `_goldsluice.json`, and the example writes to `~/goldsluice-output/`, so the common cases pass. If you really mean it, add `--allow-output-in-repo`.

**One output root.** Set the environment variable `GOLDSLUICE_OUTPUT` to a folder (for example `C:/NSDF/goldsluice-output`) and a *relative* `output` in a definition (`output = "r76-nai"`) goes under it, so the file holds no path of your machine and runs unchanged on another computer. A full path (`output = "~/goldsluice-output/r76"`) is used as written; with the variable unset a relative `output` is relative to the definition file, as before. The goldpan writers (`parquet.write`, `rq_root.write`) write exactly where they are told and do not check the folder: the guard is batch mode's.

**Steps that build on each other.** A step reads the raw trace unless it has an `input`. An algorithm that gives a *trace* instead of numbers (a filter or a baseline subtraction: `python -m goldsluice algorithms` says which: `lowpass`, `subtract_line`) needs a `name` and writes no quantity; a later step with `input = "<name>"` reads its result instead of the raw trace. A `suffix` is added to the names of the quantities of a step, so the same algorithm can run on both (`adc_max` and `adc_max_lp`). An input is a step *above* the one that reads it, and its channels must include the channels of the reader. The example `examples/r76-registry.dataset.toml` has two chains:

```toml
[[processing]]
algorithm = "lowpass"              # gives a trace: it has a name
name = "lp"
channels = "role:phonon"
[processing.params]
cutoff_hz = 95e3
order = 4
sample_period_s = "@readout.sample_period_s"

[[processing]]
algorithm = "peak"                 # reads the filtered trace
channels = "role:phonon"
input = "lp"
suffix = "_lp"                     # adc_max_lp, current_max_lp
[processing.params]
# glitch_samples, pretrigger_samples, amps_per_adc ...
```

**Run metadata.** Facts about whole series (high voltage, trigger setting) that an analysis needs for every event are written

in the definition, one `[[series]]` table per series with the same keys in each (see `examples/r76.dataset.toml`):

```
[[series]]
series = "07221124_2022"
hv_v = -34.0
trigger = "DCRC1pA 2"
trigger_value = 2.0
```

They are **joined when the data are read**: `Dataset.processed()` adds one column per key to every event (`hv_v`, `trigger_value`, ...), so a change of a value never redoes a file, and `goldsluice run` does not write them. A series with events that is not listed is an error that names it; so is a key named like a quantity or a channel; `processed(metadata=False)` leaves them out. For events you already have, `goldpan.runmeta.per_event(series, table)` gives one value per event and `groups(series, table, by=("hv_v", "trigger_value"))` the events of each chunk.

## 3. Read it back with goldpan

```
from goldpan.dataset import Dataset

ds = Dataset.open("r76.dataset.toml")
ev = ds.read()          # the pulses with the processed quantities: one Events
q = ds.processed()      # only the processed quantities, no pulses: small, fast
ev.quantities["Side2pC"]["current_max"]    # a record per channel, a field per output
```

`ds.iter_events()` gives one `Events` per raw file (for a dataset larger than memory); `ds.read(processed=False)` gives only the pulses. Both packages read the *same* file, so a notebook and a batch job cannot disagree about what the dataset is. Reading before the batch run raises an error that names the command to run.

## 4. What is written, and what makes it run again

```
~/goldsluice-output/r76/
    _goldpan.json        the goldpan Parquet manifest
    _goldsluice.json     the ledger: per raw file, what was done, with what, how long
    NAI-S2/07221203_2025_F0001.parquet    one file per series and dump:
                         the key (series, dump, event) and the quantities
```

Any other tool that reads Parquet (pandas, Polars, DuckDB) opens these files. Each carries its provenance (the steps and parameters, the goldsluice version, the timing) in its metadata under `goldsluice`. A raw file is **processed again** when its size or date changes, or when anything that determines its results changes: the way it is read, the readout, a step or a parameter. Renaming the dataset or moving the output does not count. To start over, delete the output folder or run with `--force`.

## 5. Adding steps

Add another `[[processing]]` table. Each quantity must come from **one** step (two steps giving `Side2pC.adc_max` is an error that names it); different algorithms add different fields to the same channel's record. Use `channels = ["Side2pC"]` to process only some channels. For an algorithm that does not exist yet, write its Python mirror and Rust kernel (the other sheet): `algorithms` lists it as soon as it is registered.

## 6. ROOT files instead of Parquet

Two lines in `[dataset]` write the quantities as ROOT files laid out like the collaboration's RQ files (they need `uproot`: `pip install goldsluice[root]`):

```
[dataset]
name = "r76-07221203"
output = "~/goldsluice-output/r76-root"
output_format = "root"              # "parquet" is the default
root_detector = 1                   # N of the zip<N> tree: required, never assumed
```

There is one file per series and dump, `<series>_<dump>.root`, with `rqDir/eventTree` (`SeriesNumber`,

DumpNumber, EventNumber) and rqDir/zip<N> row-aligned, every branch a double, as in the real RQ files. The branches keep **our names**, Side2pC_adc_max and not the collaboration's PAS1max: ROOT is only the container. The provenance of the run is stored as the string infoDir/goldsluice. The ledger, the chunks and the output-location guard work as for Parquet.

Read the files back with the RQ reader of goldpan (Dataset.processed() reads Parquet output only, and says so):

```
from pathlib import Path
from goldpan.adapters import rq_root

t = rq_root.read(Path.home() / "goldsluice-output" / "r76-root", detector=1)
t["Side2pC_adc_max"]        # one record per event
```

root_detector = 1 is **our choice** for the NaI detector of Run 76: the number is not in the raw data, so it lives in the definition file; use the same one in every definition of that detector. (The collaboration's Z7 files have zip7 and zip14.) The commented lines in examples/r76.dataset.toml show the same two keys.

## 7. When it fails

| Message | Meaning and fix |
|---|---|
| unknown key(s) [...] | A typo or a key that does not exist; the message lists the allowed ones. |
| ...matches nothing | A paths or glob entry found no file; the message says where it looked (relative paths start at the definition file). |
| readout needs [...] | An inline readout lacks numbers; there are no defaults. |
| needs a readout | The adapter (idx, midas) takes a readout and the source gives none. |
| has no argument [...] | [sources.read] names something the adapter's read does not take. |
| unknown algorithm | Not in algorithms; check the spelling or register the algorithm. |
| missing ... keyword-only argument | A step lacks a parameter: algorithms have no defaults. |
| nothing processed yet | You read the dataset before run. |
| has the batch run covered this file | New raw files were added after the last run; run again. |
| inside the Git working tree | The output folder is in a repository that does not ignore it: write it elsewhere (~/goldsluice-output/...), ignore it, or pass --allow-output-in-repo. |
| needs root_detector | output_format = "root" without the number of the zip<N> tree. |
| rq_root.read (from processed()) | The output is ROOT: read it with rq_root.read, not Dataset.processed(). |

The numbers in current_max are in **ADC counts times** amps_per_adc: a placeholder of 1.0, as in the example file, gives counts.